DEPARTMENT OF ROBOTICS ENGINEERING

COURSE: ROBOTICS, DYNAMICS & CONTROL (SEM 5 / III YEAR)

STEP-BY-STEP IMPLEMENTATION & SIMULATION EXECUTION MANUAL

Heterogeneous Multi-Robot Coordination (UR5 + TurtleBot3 OpenManipulator) Using ROS 2 Actions & Services in ROS 2 Jazzy Jalisco on Ubuntu 24.04 LTS

Target Operating System:	Ubuntu 24.04 LTS (Noble Numbat)
ROS 2 Distribution:	**ROS 2 Jazzy Jalisco** (Latest Tier 1 LTS Release)
Assigned Project:	Team 3: Multi-Robot Coordination Using ROS 2 Actions & Services
Designated Robots:	1) Fixed Manipulator: 6-DOF Universal Robot (UR5) 2) Mobile Manipulator: TurtleBot3 (Waffle Pi) + OpenManipulator-X
Simulation Platforms:	Gazebo Harmonic (ros_gz), RViz2, Nav2, MoveIt 2, ros2_control

ABOUT THIS PRACTICAL GUIDE:
This guide is written specifically for students starting from scratch on Ubuntu Linux with ROS 2 Jazzy. Every command is presented in copy-paste ready format, explaining what each terminal does, how the simulation behaves, and how to verify that your ROS 2 Actions and Services are working properly.

Phase 1: Ubuntu 24.04 & ROS 2 Jazzy Prerequisites Setup

Before building the multi-robot project, ensure your Ubuntu 24.04 environment has all necessary ROS 2 Jazzy navigation, manipulation, and simulation libraries installed.

Step 1.1: Verify ROS 2 Jazzy Installation

Open a terminal (Ctrl + Alt + T) and verify that ROS 2 Jazzy is active:

```
source /opt/ros/jazzy/setup.bash
echo "Active ROS Distro: $ROS_DISTRO"
# Expected Output: Active ROS Distro: jazzy
```

Step 1.2: Install Required ROS 2 Jazzy Packages

Run the following commands to install Nav2, MoveIt 2, Gazebo bridge, and robot packages:

```
sudo apt update
sudo apt install -y \
  ros-jazzy-navigation2 \
  ros-jazzy-nav2-bringup \
  ros-jazzy-moveit \
  ros-jazzy-moveit-ros-planning-interface \
  ros-jazzy-ros2-control \
  ros-jazzy-ros2-controllers \
  ros-jazzy-ros-gz \
  ros-jazzy-turtlebot3 \
  ros-jazzy-turtlebot3-msgs \
  ros-jazzy-turtlebot3-simulations \
  ros-jazzy-dynamixel-sdk \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-pip
```

Step 1.3: Initialize and Update Rosdep

```
sudo rosdep init 2>/dev/null || true
rosdep update
```

Phase 2: Setting Up the Multi-Robot Colcon Workspace

Follow these steps to create your ROS 2 workspace and configure the multi_robot_coordination package.

Step 2.1: Create Workspace Directory Structure

```
mkdir -p ~/ros2_ws/src/multi_robot_coordination/action
mkdir -p ~/ros2_ws/src/multi_robot_coordination/srv
mkdir -p ~/ros2_ws/src/multi_robot_coordination/multi_robot_coordination
mkdir -p ~/ros2_ws/src/multi_robot_coordination/launch
mkdir -p ~/ros2_ws/src/multi_robot_coordination/config
cd ~/ros2_ws/src/multi_robot_coordination
```

Step 2.2: Copy Project Files from Workspace to Ubuntu

Copy all provided Python nodes, action interfaces, service interfaces, launch files, and package manifests from the project repository into ~/ros2_ws/src/multi_robot_coordination/.

Directory check: verify that your folder layout matches:

```
~/ros2_ws/src/multi_robot_coordination/
├── action/
│   ├── NavigateAndPick.action
│   └── UR5PickAndPlace.action
├── srv/
│   ├── AcquireHandoffLock.srv
│   └── TriggerGripper.srv
├── multi_robot_coordination/
│   ├── __init__.py
│   ├── scheduler.py
│   ├── simulation_runner.py
│   ├── multi_robot_coordinator.py
│   ├── tb3_action_server.py
│   ├── ur5_action_server.py
│   └── handoff_service_server.py
├── launch/
│   └── multi_robot_system.launch.py
├── package.xml
└── setup.py
```

Phase 3: Building and Sourcing the Package

Compile the workspace using colcon and source the overlay environment.

Step 3.1: Build Workspace with Colcon

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
```

Step 3.2: Source the Built Workspace Overlay

```
source install/setup.bash
# To make sourcing permanent in your terminal:
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

Step 3.3: Verify Installed Interfaces

```
ros2 interface show multi_robot_coordination/action/NavigateAndPick
ros2 interface show multi_robot_coordination/srv/AcquireHandoffLock
```

Phase 4: Step-by-Step Execution Guide (Terminal by Terminal)

To run the complete coordinated multi-robot simulation, open multiple terminal windows or use tmux / terminator tabs. Follow the exact sequence below:

TERMINAL 1: Shared Transfer Zone Lock Service Server

```
Launches the mutual exclusion lock server that manages exclusive access to the transfer zone:
source ~/ros2_ws/install/setup.bash
ros2 run multi_robot_coordination lock_server
# Expected: [INFO] [handoff_lock_server]: Shared Transfer Zone Lock Service Server Active.
```

TERMINAL 2: TurtleBot3 Mobile Manipulator Action Server

```
Launches the Action Server simulating mobile base navigation and OpenManipulator-X grasping:
source ~/ros2_ws/install/setup.bash
ros2 run multi_robot_coordination tb3_server
# Expected: [INFO] [tb3_action_server]: TurtleBot3 Mobile Manipulator Action Server Started.
```

TERMINAL 3: UR5 Fixed Manipulator Action Server

```
Launches the Action Server executing 6-DOF MoveIt 2 trajectory planning and assembly:
source ~/ros2_ws/install/setup.bash
ros2 run multi_robot_coordination ur5_server
# Expected: [INFO] [ur5_action_server]: UR5 6-DOF Manipulator Action Server Started.
```

TERMINAL 4: Central Multi-Robot Coordinator Node

```
Launches the main orchestrator with Priority-Based scheduling:
source ~/ros2_ws/install/setup.bash
ros2 run multi_robot_coordination coordinator --ros-args -p scheduler_mode:=PRIORITY
# Expected: [INFO] [multi_robot_coordinator]: Task Scheduler Mode set to: PRIORITY
```

TERMINAL 5: (All-in-One Alternative) Launching Entire System via Launch File

```
Instead of opening 4 individual terminals, launch everything at once using the master launch file:
source ~/ros2_ws/install/setup.bash
ros2 launch multi_robot_coordination multi_robot_system.launch.py
```

Phase 5: Interactive Testing, Introspection & Monitoring

Open a new terminal to inspect the active ROS 2 Actions, Services, Topics, and test triggering operations manually:

5.1: Inspect Active ROS 2 Actions & Services

```
# List all active Action Servers
ros2 action list
# Output: /navigate_and_pick
#         /ur5_pick_and_assemble

# List all active Services
ros2 service list | grep -E 'lock|gripper'
# Output: /acquire_transfer_lock
```

5.2: Send a Manual Test Goal to TurtleBot3 Action Server

```
ros2 action send_goal --feedback /navigate_and_pick multi_robot_coordination/action/NavigateAndPick \
  "{target_station_id: 'STATION_A', pickup_coordinates: [1.2, 0.8, 0.15], priority_level: 1}"
# You will see real-time 10 Hz feedback streamed to your terminal until completion!
```

5.3: Test Shared Zone Lock Service

```
# Request lock acquisition:
ros2 service call /acquire_transfer_lock multi_robot_coordination/srv/AcquireHandoffLock \
  "{robot_id: 'turtlebot3', zone_id: 1, request_lock: true}"

# Release lock:
ros2 service call /acquire_transfer_lock multi_robot_coordination/srv/AcquireHandoffLock \
  "{robot_id: 'turtlebot3', zone_id: 1, request_lock: false}"
```

Phase 6: Running the Automated Benchmark Suite & Generating Results

To evaluate and reproduce the FIFO, Priority-Based, and Round-Robin benchmark metrics reported in the project submission:

```
cd ~/ros2_ws/src/multi_robot_coordination/multi_robot_coordination
python3 simulation_runner.py
```

Expected Terminal Output:
The script will run 30 stochastic assembly jobs across all 3 algorithms and print the exact metrics table (Makespan, Avg Wait Time, Priority-1 Wait Time, UR5 Utilization %, TB3 Utilization %, and Tasks/Hour).

Phase 7: ROS 2 Jazzy Specific Troubleshooting Tips

Issue: colcon build fails with rosidl_default_generators not found
Fix: Run sudo apt install ros-jazzy-rosidl-default-generators ros-jazzy-rosidl-default-runtime and re-run colcon build.

Issue: ros2: command not found or action interfaces cannot be imported in Python
Fix: Make sure you ran source /opt/ros/jazzy/setup.bash followed by source ~/ros2_ws/install/setup.bash in EVERY open terminal.

Issue: Action Server reports wait_for_server() timeout
Fix: Check if the nodes are running under different ROS_DOMAIN_ID. In each terminal run: export ROS_DOMAIN_ID=42 and restart the nodes.

Issue: Permission denied when running Python node scripts
Fix: Give executable permissions to all Python files: chmod +x ~/ros2_ws/src/multi_robot_coordination/multi_robot_coordination/*.py.